Visual Studio Code con Python

UD2 · Entornos de Desarrollo · 1.º DAW · Guía, ejercicio y laboratorio de errores

Índice

1 Antes de empezar: dos aclaraciones **2**

2 PARTE 1 · Guía paso a paso **3**

2.1 1.1 · Instalar Python . . . 3
2.2 1.2 · Instalar Visual Studio Code . . . 3
2.3 1.3 · Las extensiones . . . 3
2.4 1.4 · Abrir una CARPETA, no un fichero . . . 4
2.5 1.5 · Seleccionar el intérprete . . . 4
2.6 1.6 · Tu primer programa . . . 4
2.7 1.7 · Las tres formas de ejecutarlo . . . 5
2.8 1.8 · Terminal y consola de depuración: no son lo mismo . . . 5
2.9 1.9 · Pedir datos al usuario . . . 5
2.10 1.10 · El depurador . . . 6
2.11 1.11 · Formateo automático . . . 7
2.12 1.12 · Los avisos: subrayados y panel de Problemas . . . 8
2.13 1.13 · Atajos que hay que saberse . . . 9
2.14 1.14 · Cuando algo no funciona . . . 9

3 PARTE 2 · Ejercicio evaluable P2.1 **10**

3.1 Fase 0 · El entorno (3 capturas) . . . 10
3.2 Fase 1 · Primer programa (1 captura) . . . 10
3.3 Fase 2 · La trampa del input (2 capturas) . . . 10
3.4 Fase 3 · Depuración (5 capturas) — *la fase que más puntúa* . . . 11
3.5 Fase 4 · Formateo (2 capturas) . . . 12
3.6 Fase 5 · Punto de ruptura condicional (2 capturas) . . . 12
3.7 Fase 6 · Memoria final . . . 12
3.8 Qué se entrega . . . 13

4 PARTE 3 · Laboratorio de errores **14**

4.1 Bloque A · Errores que impiden ejecutar . . . 14
4.2 Bloque B · Errores lógicos: se ejecuta, pero está mal . . . 16

1 Antes de empezar: dos aclaraciones

Ojo

1. Visual Studio Code no es Visual Studio. Son dos programas distintos de Microsoft:

	Visual Studio Code	Visual Studio
Qué es	Un **editor** muy ampliable con extensiones	Un **IDE** completo
Tamaño	~100 MB	Varios GB
Sistemas	Windows, macOS, Linux	Windows (y una versión para Mac ya retirada)
Lenguajes	Los que le añadas con extensiones	Sobre todo C#, C++, .NET
Precio	Gratuito	Hay versión gratuita (Community) y de pago

Nosotros usamos **Visual Studio Code**, VS Code para abreviar. Cuando busquéis ayuda en internet, poned «VS Code»: si ponéis «Visual Studio» os saldrán respuestas del otro programa y no os funcionarán.

2. Python no se compila: se ejecuta. Vais a ver el botón ▷ y vais a oír a gente decir «compilar». Con Python **no hay compilación**: no se genera ningún ejecutable. Lo que ocurre es que el **intérprete** de Python lee vuestro fichero `.py` y lo va ejecutando. Es exactamente lo que vimos en la UD1, y aquí lo vais a ver funcionando.

2 PARTE 1 · Guía paso a paso

2.1 1.1 · Instalar Python

Windows

1. Entra en `python.org` → *Downloads* → el botón grande de descarga.
2. Ejecuta el instalador y, **antes de pulsar Install**, marca la casilla de abajo:
   ☐ **Add python.exe to PATH**
   Es la casilla más importante de toda la instalación. Si no la marcas, la terminal no encontrará Python y todo lo demás fallará.
3. *Install Now* y espera.

macOS y Linux suelen traer Python. Compruébalo antes de instalar nada.

Comprobación. Abre una terminal (en Windows, tecla Windows → escribe `cmd` → Enter) y escribe:

```
python --version
```

Tiene que responder algo como `Python 3.13.1`. Necesitas **3.12 o superior**.

Ojo

Si responde «no se reconoce como un comando interno o externo», es que no marcaste la casilla del PATH. Solución: vuelve a ejecutar el instalador, elige *Modify* y márcala. **Reinicia la terminal después**: las terminales ya abiertas no se enteran del cambio.

2.2 1.2 · Instalar Visual Studio Code

Desde `code.visualstudio.com`. Durante la instalación, marca estas dos casillas:

☐ Agregar la acción «Abrir con Code» al menú contextual de **archivos** ☐ Agregar la acción «Abrir con Code» al menú contextual de **directorios**

Te van a ahorrar mucho tiempo: con ellas puedes hacer clic derecho en cualquier carpeta y abrirla directamente en VS Code.

2.3 1.3 · Las extensiones

VS Code recién instalado **no sabe nada de Python**. Es un editor de texto con colores. Todo lo que vamos a hacer —ejecutar, depurar, formatear— lo aportan las extensiones.

Se instalan desde el icono de los cuatro cuadraditos de la barra izquierda, o con `Ctrl+Mayús+X`. Busca cada una por su nombre y pulsa *Install*.

Extensión	Quién la publica	Para qué sirve
Python	Microsoft	La base: ejecutar, seleccionar intérprete, terminal
Pylance	Microsoft	Autocompletado, detección de errores mientras escribes, ir a la definición
Python Debugger	Microsoft	El depurador: puntos de ruptura, variables, paso a paso
Ruff	Astral Software	Formateo automático y avisos de código sospechoso
Spanish Language Pack	Microsoft	*Opcional*: pone la interfaz en español

Idea clave

Las tres primeras se instalan **juntas**: al instalar «Python», VS Code arrastra Pylance y Python Debugger automáticamente. Compruébalo en la lista de extensiones instaladas; si alguna falta, instálala a mano. **Ruff** hay que instalarla aparte. Antes se usaban dos herramientas distintas —Black para formatear y Flake8 para avisar— y Ruff hace las dos cosas y va mucho más rápido.

2.4 1.4 · Abrir una CARPETA, no un fichero

Esto es lo que más gente hace mal el primer día.

VS Code trabaja con **carpetas de proyecto**, no con ficheros sueltos. Si abres un `.py` suelto, la mitad de las funciones no estarán disponibles y el depurador se quejará.

1. Crea una carpeta en tu disco, por ejemplo `C:\DAW\ENDES\UD2`.
2. En VS Code: *Archivo → Abrir carpeta…* y selecciónala.
3. En la barra izquierda, el primer icono (*Explorador*) te muestra el contenido de esa carpeta.

A partir de ahora, todos los ficheros los creas ahí dentro con el icono de «nuevo archivo» del Explorador.

Ojo

Nada de acentos, eñes ni espacios en las rutas. `C:\DAW\ENDES\UD2` va bien. `C:\Mis Documentos\Programación\Módulo Entornos` te va a dar problemas raros y difíciles de diagnosticar. Es una costumbre profesional: acostúmbrate desde hoy.

2.5 1.5 · Seleccionar el intérprete

Hay que decirle a VS Code **qué Python** tiene que usar. En un ordenador puede haber varios.

1. `Ctrl`+`Mayús`+`P` abre la **paleta de comandos**. Es la forma de llegar a todo en VS Code: si no encuentras algo en los menús, está aquí.
2. Escribe `Python: Select Interpreter` y pulsa Enter.
3. Elige el que tenga la versión más alta.

Cuando esté seleccionado, verás la versión en la **barra de estado**, abajo del todo a la derecha. Si ahí no pone nada, el intérprete no está puesto y no vas a poder ejecutar.

2.6 1.6 · Tu primer programa

Crea un fichero llamado `hola.py` y escribe dentro:

```
AUTOR = "Vázquez Carreño"

print("Hola, soy", AUTOR)
print("Este es mi primer programa en Python")
```

Idea clave

Qué es cada cosa, porque todavía no habéis dado Python:

- `AUTOR = "..."` crea una **variable** llamada AUTOR y le mete dentro un texto. Las comillas marcan dónde empieza y dónde acaba el texto.
- `print(...)` **escribe en pantalla** lo que le pongas entre paréntesis.
- Si le pones varias cosas separadas por comas, las escribe seguidas con un espacio entre medias.

2.7 1.7 · Las tres formas de ejecutarlo

Forma	Cómo	Cuándo se usa
El botón ▷	Arriba a la derecha	El día a día
Ejecutar sin depurar	`Ctrl+F5`	Lo mismo, sin tocar el ratón
Desde la terminal	`python hola.py`	Cuando quieres ver qué hace de verdad

La tercera merece que la hagas al menos una vez. Abre la terminal integrada con `Ctrl+Ñ` (o *Terminal* → *Nueva terminal*) y escribe:

```
python hola.py
```

Eso es **exactamente** lo que hace el botón ▷ por debajo. El botón no es magia: escribe ese comando por ti. Verlo una vez evita muchos malentendidos.

2.8 1.8 · Terminal y consola de depuración: no son lo mismo

Abajo tienes varias pestañas y conviene no confundirlas:

Pestaña	Qué es
Terminal	Una línea de comandos de verdad. Aquí sale lo que imprime tu programa al ejecutarlo normal
Consola de depuración	Solo aparece mientras depuras. Además de la salida, **puedes escribir expresiones** y te las evalúa al vuelo
Problemas	Los avisos que encuentran Pylance y Ruff sin ejecutar nada
Salida	Los mensajes internos de VS Code y de las extensiones. Casi nunca te hace falta

2.9 1.9 · Pedir datos al usuario

Modifica `hola.py`:

```python
AUTOR = "Vázquez Carreño"

nombre = input("¿Cómo te llamas? ")
print("Hola,", nombre, "- programa de", AUTOR)
```

`input(...)` **para el programa**, escribe el mensaje y espera a que escribas algo y pulses Enter. Lo que escribas se guarda en la variable.

Ojo

La trampa número uno de Python, y la vas a pisar hoy: `input()` **siempre devuelve texto**, aunque escribas un número. Si escribes 5, en la variable no hay el número cinco: hay el texto «5».
Por eso esto falla:

```python
edad = input("Edad: ")
print(edad + 1)        # TypeError
```

Y esto funciona:

```python
edad = int(input("Edad: "))
print(edad + 1)
```

`int(...)` convierte texto a número entero. Para decimales se usa `float(...)`. En la Parte 3 tienes

el caso completo con su mensaje de error.

2.10 1.10 · El depurador

Aquí está lo que de verdad justifica usar un IDE en lugar del Bloc de notas.

Hasta ahora, para saber qué valía una variable tenías que poner un `print`. El depurador te deja **parar el programa donde quieras y mirar dentro**.

2.10.1 Poner un punto de ruptura

Haz clic en el **margen izquierdo**, justo a la izquierda del número de línea. Aparece un **punto rojo**. También puedes ponerlo con F9 sobre la línea donde esté el cursor.

Un punto de ruptura significa: *«cuando llegues aquí, párate y avísame»*.

2.10.2 Arrancar la depuración

Pulsa **F5**. La primera vez te preguntará qué tipo de depuración quieres: elige **Python File**.

El programa arranca y se detiene en tu punto de ruptura. La línea se resalta: **esa línea todavía no se ha ejecutado**, está a punto de hacerlo. Es un detalle que confunde mucho al principio.

2.10.3 Los cuatro paneles

Cuando el programa está parado, el panel izquierdo cambia:

Panel	Qué te enseña
Variables	Todas las variables vivas en este momento y su valor. Es el panel estrella
Inspección (*Watch*)	Expresiones que tú eliges vigilar. Puedes escribir `nota * 2` y ver el resultado en cada paso
Pila de llamadas	Qué función ha llamado a cuál para llegar hasta aquí
Puntos de interrupción	La lista de todos los que has puesto, para activarlos o quitarlos

2.10.4 La barra de control

Aparece flotando arriba. Seis botones, y estos son los que importan:

Tecla	Botón	Qué hace
`F5`	**Continuar**	Sigue a toda velocidad hasta el siguiente punto de ruptura
`F10`	**Paso a paso por procedimientos**	Ejecuta la línea actual **entera** y pasa a la siguiente
`F11`	**Paso a paso por instrucciones**	Si la línea llama a una función, **entra dentro** de ella
`Mayús+F11`	**Salir**	Termina la función actual y vuelve a quien la llamó
`Mayús+F5`	**Detener**	Corta la ejecución

Idea clave

La diferencia entre F10 y F11 es la que más cuesta, y es sencilla:
Imagina que estás en la línea `media = calcular_media(notas)`.

- Con **F10**, la función se ejecuta entera de golpe y tú apareces en la línea siguiente, con el resultado

ya calculado. *No has visto qué ha pasado dentro.*
- Con **F11**, saltas **dentro** de `calcular_media` y la recorres línea a línea.

Regla práctica: **F10 mientras buscas dónde está el problema, F11 cuando ya sabes en qué función está.**

2.10.5 Cambiar un valor sobre la marcha

En el panel **Variables**, haz doble clic sobre el valor de una variable y escribe otro. El programa seguirá con el valor nuevo.

Sirve para probar un caso raro sin tener que reescribir el código: «¿y si aquí llegara un cero?».

2.10.6 Puntos de ruptura condicionales

Si tienes un bucle de 500 vueltas y el fallo está en la 400, no vas a pulsar F10 cuatrocientas veces.

Clic derecho sobre el punto rojo → *Editar punto de interrupción…* y escribe una condición:

```
nota > 10
```

El programa solo se parará cuando esa condición sea cierta. Es de lo más útil que tiene el depurador y casi nadie lo usa.

2.10.7 Punto de registro (*logpoint*)

Clic derecho en el margen → **Agregar punto de registro**. En vez de parar, **escribe un mensaje** en la consola de depuración. Lo que pongas entre llaves se sustituye por su valor:

```
Vuelta {i}, la nota vale {nota}
```

Es como poner un `print`, **pero sin tocar el código**. Cuando termines, lo quitas y no queda rastro. Los `print` de depuración que se quedan olvidados en el código son un clásico: esto lo evita.

2.11 1.11 · Formateo automático

Escribir el código ordenado no es cuestión de estética: es lo que permite que otra persona —o tú dentro de tres meses— lo lea sin sufrir.

Lo bueno es que **no hay que hacerlo a mano**.

2.11.1 Activarlo

`Ctrl+,` abre la configuración. Arriba a la derecha hay un icono de un documento con una flecha: *Abrir configuración (JSON)*. Pégale esto dentro de las llaves:

```
{
  "editor.formatOnSave": true,
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff"
  }
}
```

A partir de ahora, **cada vez que guardes con `Ctrl+S`, el código se ordena solo**.

2.11.2 Qué hace exactamente

Esto es código real, escrito mal a propósito:

```
AUTOR="Vazquez Carreno"
import math
def area_circulo( radio ):
    resultado=math.pi*radio**2
    return resultado
def  area_rectangulo(base,altura):
        return base*altura
print( "Autor:",AUTOR )
```

Al guardar, Ruff lo deja así:

```
AUTOR = "Vazquez Carreno"
import math


def area_circulo(radio):
    resultado = math.pi * radio**2
    return resultado


def area_rectangulo(base, altura):
    return base * altura


print("Autor:", AUTOR)
```

Espacios alrededor de los = y de los operadores, sin espacios sobrantes dentro de los paréntesis, dos líneas en blanco entre funciones, sangría uniforme.

Ojo

El formateador no arregla errores. Si tu programa está mal, después de formatearlo seguirá mal, pero más bonito. Son dos cosas distintas:

- El **formateador** (Ruff) cambia cómo se ve el código.
- El **analizador** (Pylance y Ruff) te avisa de cosas sospechosas: una variable que declaras y no usas, un nombre mal escrito, un `import` que sobra.
- El **depurador** te sirve para encontrar por qué el programa hace algo distinto de lo que quieres.

2.11.3 Formatear sin guardar

`Mayús+Alt+F` formatea el documento en cualquier momento.

2.12 1.12 · Los avisos: subrayados y panel de Problemas

Mientras escribes, van apareciendo subrayados:

Color	Qué significa	Ejemplo
Rojo	Error: esto no va a funcionar	Falta un paréntesis, variable inexistente
Amarillo	Aviso: funciona, pero es mejorable	Variable declarada y nunca usada
Azul claro	Sugerencia	Se puede escribir de forma más simple

Pon el ratón encima del subrayado y te explica qué pasa. La pestaña **Problemas** (`Ctrl+Mayús+M`) te los junta todos en una lista.

2.13 1.13 · Atajos que hay que saberse

Atajo	Qué hace
`Ctrl+Mayús+P`	Paleta de comandos. **El más importante de todos**
`Ctrl+S`	Guardar (y formatear, si lo has activado)
`Ctrl+F5`	Ejecutar sin depurar
`F5`	Empezar a depurar
`F9`	Poner o quitar punto de ruptura
`F10` / `F11`	Paso a paso por encima / por dentro
`Ctrl+Ñ`	Abrir o cerrar la terminal
`Ctrl+B`	Mostrar u ocultar la barra lateral
`Ctrl+/`	Comentar o descomentar la línea
`Alt+↑` / `Alt+↓`	Mover la línea arriba o abajo
`Mayús+Alt+↓`	Duplicar la línea
`Ctrl+D`	Seleccionar la siguiente aparición de lo seleccionado
`F2`	Renombrar una variable **en todo el fichero**

Idea clave

F2 es el que más sorprende. Pon el cursor sobre el nombre de una variable, pulsa F2, escribe el nombre nuevo y pulsa Enter: se cambia en todos los sitios donde aparecía, y solo donde era esa variable. No es un buscar y reemplazar: entiende el código. En la UD4 veremos que eso tiene un nombre, **refactorizar**.

2.14 1.14 · Cuando algo no funciona

Síntoma	Causa casi segura	Solución
«python no se reconoce como comando»	No marcaste *Add to PATH*	Reinstala marcándola y **reinicia la terminal**
El botón ▷ no aparece	No has seleccionado intérprete	`Ctrl+Mayús+P` → *Python: Select Interpreter*
El depurador no arranca	Abriste un fichero suelto, no la carpeta	*Archivo* → *Abrir carpeta*
No formatea al guardar	Falta Ruff o la configuración	Revisa el apartado 1.11
El punto rojo sale hueco	Esa línea no se puede ejecutar (está vacía o es un comentario)	Ponlo en una línea con código
Todo subrayado en rojo de repente	Se cayó Pylance	`Ctrl+Mayús+P` → *Developer: Reload Window*

3 PARTE 2 · Ejercicio evaluable P2.1

Entrega individual. Se sube a Moodle.

Vas a montar el entorno desde cero y a demostrar que sabes usarlo. El código es lo de menos: lo que se evalúa es **el manejo de la herramienta**.

Ojo

Tus apellidos tienen que aparecer en tres sitios, y sin eso la tarea no se corrige:

1. En el **nombre de todos los ficheros**: `calculadora_perez_gomez.py`.
2. En una **constante `AUTOR`** dentro de cada programa, que se imprime al arrancar.
3. Por tanto, en **la salida por pantalla** que se ve en todas tus capturas.

Escribe los apellidos **sin acentos y en minúscula** en los nombres de fichero (`vazquez_carreno`), y como quieras dentro del código.

Las capturas deben ser de la **ventana completa de VS Code**, no recortes del código. Tienen que verse la barra lateral, el código y el panel de abajo. Pégalas en un documento y numéralas.

3.1 Fase 0 · El entorno (3 capturas)

1. La terminal con `python --version` ejecutado y su respuesta.
2. La lista de extensiones instaladas, donde se vean Python, Pylance, Python Debugger y Ruff.
3. La barra de estado abajo a la derecha con la versión del intérprete seleccionada.

3.2 Fase 1 · Primer programa (1 captura)

Crea `saludo_APELLIDOS.py`:

```
AUTOR = "Pon aquí tus apellidos"

print("Programa de:", AUTOR)
nombre = input("¿Cómo te llamas? ")
print("Encantado,", nombre)
```

Ejecútalo **desde la terminal** con `python saludo_APELLIDOS.py`, escribe tu nombre y captura la pantalla con el comando y la respuesta visibles.

3.3 Fase 2 · La trampa del input (2 capturas)

Escribe `edad_APELLIDOS.py` **tal cual**, con el error incluido:

```
AUTOR = "Pon aquí tus apellidos"

print("Programa de:", AUTOR)
edad = input("¿Cuántos años tienes? ")
print("El año que viene tendrás", edad + 1)
```

1. Ejecútalo y **captura el error completo** que sale en la terminal.
2. Escribe en tu documento, con tus palabras: **qué dice el error y por qué ocurre**.
3. Corrígelo y captura la ejecución correcta.

3.4 Fase 3 · Depuración (5 capturas) — *la fase que más puntúa*

Copia este programa **tal cual** en `notas_APELLIDOS.py`. **Tiene tres fallos**. Ninguno da error: el programa se ejecuta perfectamente y da un resultado **equivocado**.

```
AUTOR = "Pon aquí tus apellidos"


def calcular_media(lista):
    suma = 0
    for nota in lista:
        suma = nota
    media = suma / 3
    return media


def esta_aprobado(media):
    if media > 5:
        return True
    else:
        return False


notas = [7, 4, 5, 9]
media = calcular_media(notas)
print("Autor:", AUTOR)
print("Notas:", notas)
print("Media:", media)
if esta_aprobado(media):
    print("APROBADO")
else:
    print("SUSPENSO")
```

Antes de tocar nada, calcula a mano la media de 7, 4, 5 y 9. Apunta el resultado: eso es lo que el programa **debería** dar.

Ahora encuéntralos **con el depurador**. No vale mirando el código fijamente hasta que se te ocurra: hay que usar la herramienta y demostrarlo.

1. Pon un punto de ruptura **dentro del bucle**, en la línea `suma = nota`.
2. Arranca con `F5` y avanza con `F10` vuelta a vuelta.
3. **Captura el panel de Variables** en la segunda y en la tercera vuelta, de forma que se vea cómo evoluciona `suma`. ¿Está sumando o está haciendo otra cosa?
4. Usa `F11` para **entrar dentro** de `esta_aprobado` y captura la pila de llamadas.
5. Añade a **Inspección** (*Watch*) la expresión `suma / len(lista)` y captura lo que vale.

En tu documento, por cada fallo:

	Qué poner
Dónde está	Número de línea
Qué hace mal	Explicado con tus palabras
Cómo lo has encontrado	Qué te enseñó el depurador exactamente
Corrección	La línea corregida

Y al final, el programa arreglado más una captura de su ejecución correcta.

Idea clave

Pista para el tercer fallo, que es el más escurridizo: prueba el programa con las notas `[5, 5, 5]`. La media sale exactamente 5. ¿Qué dice el programa? ¿Qué debería decir?

3.5 Fase 4 · Formateo (2 capturas)

Crea `figuras_APELLIDOS.py` copiando esto **exactamente igual de mal escrito**:

```
AUTOR="Pon aqui tus apellidos"
import math
def area_circulo( radio ):
    resultado=math.pi*radio**2
    return resultado
def  area_rectangulo(base,altura):
        return base*altura
print( "Autor:",AUTOR )
print("Circulo:" ,area_circulo(3))
print("Rectangulo:",area_rectangulo( 4,5 ))
```

1. **Captura el antes.**
2. Guarda con `Ctrl+S` (con el formateo automático activado) y **captura el después**.
3. Escribe **cinco cambios concretos** que ha hecho Ruff.
4. Responde: ¿ha cambiado en algo **lo que hace** el programa? ¿Cómo lo has comprobado?

3.6 Fase 5 · Punto de ruptura condicional (2 capturas)

Crea `revision_APELLIDOS.py`:

```
AUTOR = "Pon aquí tus apellidos"

notas = [6, 7, 5, 8, 4, 9, 13, 6, 7, 5, 8, 4, 9, 6, 7]

print("Autor:", AUTOR)
contador = 0
for nota in notas:
    contador = contador + 1
    print("Alumno", contador, "->", nota)
```

En esa lista hay **una nota imposible**. Encuéntrala **sin leer la lista**:

1. Pon un punto de ruptura en la línea del `print`.
2. Clic derecho sobre él → *Editar punto de interrupción…* → escribe la condición `nota > 10`.
3. Arranca con F5. Debe parar **una sola vez**.
4. **Captura** el panel de Variables en ese momento, donde se vean `nota` y `contador`.
5. Añade ahora un **punto de registro** que escriba `Alumno {contador} tiene {nota}` y captura la consola de depuración con la lista completa.

3.7 Fase 6 · Memoria final

Un documento de **una página** respondiendo:

1. ¿Qué diferencia hay entre ejecutar con `Ctrl+F5` y con `F5`?

2. ¿Qué diferencia hay entre `F10` y `F11`? Pon un ejemplo de esta práctica.
3. ¿En qué caso concreto de esta práctica te habría costado mucho más encontrar el fallo usando solo `print`?
4. Menciona **tres** cosas que hace VS Code y que no haría el Bloc de notas.
5. ¿Has usado inteligencia artificial en esta práctica? Di dónde y para qué.

3.8 Qué se entrega

Un único documento **PDF** con todo lo anterior —capturas numeradas y respuestas— **más los cinco ficheros `.py`** en un ZIP. El PDF, con tus apellidos en el nombre.

4 PARTE 3 · Laboratorio de errores

Once programas que fallan. Cópialos, ejecútalos, **lee el mensaje** y solo después mira la corrección.

Idea clave

Por qué esto es la parte más útil del documento. Programar no consiste en escribir código correcto a la primera: consiste en leer mensajes de error y saber qué te están diciendo. Los mensajes de Python son bastante buenos si te acostumbras a mirarlos con calma.
Los mensajes se leen de abajo arriba. La última línea dice *qué* ha pasado. Las de arriba dicen *dónde*.

4.1 Bloque A · Errores que impiden ejecutar

4.1.1 Caso 1 · Falta algo en la línea

```
nota = 7
if nota >= 5
    print("Aprobado")
```

```
  File "caso1.py", line 2
    if nota >= 5
               ^
SyntaxError: expected ':'
```

Qué dice: «esperaba dos puntos». Un `SyntaxError` es que Python **ni siquiera ha entendido** lo que has escrito, así que no ha ejecutado nada en absoluto.

Corrección: `if nota >= 5:` — en Python, `if`, `else`, `for`, `while` y `def` terminan siempre en dos puntos.

4.1.2 Caso 2 · La sangría

```
nota = 7
if nota >= 5:
print("Aprobado")
```

```
  File "caso2.py", line 3
    print("Aprobado")
    ^
IndentationError: expected an indented block after 'if' statement on line 2
```

Qué dice: «esperaba un bloque con sangría después del if». En casi todos los lenguajes los bloques se marcan con llaves; **en Python se marcan con la sangría**, y por eso es obligatoria.

Corrección: cuatro espacios antes del `print`. Con el formateo automático activado, esto se arregla solo al guardar.

4.1.3 Caso 3 · Una letra de menos

```
nombre = "Ana"
print("Hola " + nombe)
```

```
NameError: name 'nombe' is not defined. Did you mean: 'nombre'?
```

Qué dice: «no sé qué es `nombe`». Y encima te sugiere lo que querías escribir.

Corrección: `nombre`. Fíjate en que **Pylance te lo subraya en rojo antes de ejecutar**: este error no debería llegar nunca a la ejecución.

4.1.4 Caso 4 · Texto más número — *el importante*

```
edad = input("Edad: ")
print(edad + 1)
```

```
TypeError: can only concatenate str (not "int") to str
```

Qué dice: «solo puedo concatenar texto con texto». `input()` devuelve **texto**, aunque escribas un número, y Python no suma un texto y un número.

Corrección:

```
edad = int(input("Edad: "))
print(edad + 1)
```

Ojo

Este es **el error más frecuente de todo el curso**. Cada vez que pidas un número por teclado, acuérdate de envolverlo: `int(...)` para enteros, `float(...)` para decimales.

4.1.5 Caso 5 · Dividir por cero

```
alumnos = 0
total = 30
print(total / alumnos)
```

```
ZeroDivisionError: division by zero
```

Qué dice: lo que pone. Ojo: **el programa ya estaba funcionando** cuando ha reventado. No es un error de escritura, es un error que depende de los datos.

Corrección: comprobar antes.

```
if alumnos > 0:
    print(total / alumnos)
else:
    print("No hay alumnos")
```

4.1.6 Caso 6 · Convertir lo inconvertible

```
n = int("siete")
print(n)
```

```
ValueError: invalid literal for int() with base 10: 'siete'
```

Qué dice: «eso no es un número en base 10». El tipo de dato es el correcto (texto), pero el **valor** no vale.

Corrección: `int("7")`. Y cuando el texto venga de un `input()`, hay que contar con que el usuario escriba cualquier cosa.

4.2 Bloque B · Errores lógicos: se ejecuta, pero está mal

Ojo

Estos son mucho peores que los anteriores. Los del bloque A te los canta el traductor. Estos no: el programa arranca, no da ningún error y devuelve un resultado equivocado. Nadie te avisa.
Aquí es donde el depurador deja de ser una comodidad y pasa a ser imprescindible.

4.2.1 Caso 7 · La calculadora que resta

```
a = 8
b = 5
print("Suma:", a - b)
```

Salida: `Suma: 3` — sin ningún error.

Qué pasa: pone «Suma» y resta. Un descuido de un solo carácter.

Corrección: `a + b`, que da 13.

Cómo se detecta: probando con un caso cuyo resultado conoces. 8 + 5 son 13 y salen 3. Por eso **siempre hay que probar el programa con datos de los que sepas la respuesta**. Es la idea de la que va la UD3 entera.

4.2.2 Caso 8 · La media que no es la media

```
n1 = 8
n2 = 6
print("Media:", n1 + n2 / 2)
```

Salida: `Media: 11.0`, cuando la media de 8 y 6 es 7.

Qué pasa: la división se hace **antes** que la suma. El programa calcula `8 + (6/2)`.

Corrección: `(n1 + n2) / 2`.

Cómo se detecta: un punto de ruptura y la expresión `n2 / 2` en el panel de Inspección. Se ve inmediatamente que vale 3 y no 7.

4.2.3 Caso 9 · La condición al revés

```
nota = 3
if nota < 5:
    print("Aprobado")
else:
    print("Suspenso")
```

Salida: `Aprobado`, con un 3.

Qué pasa: el signo está al revés.

Corrección: `if nota >= 5:`.

Cómo se detecta: probando **los dos casos**. Con una sola prueba habría parecido correcto.

4.2.4 Caso 10 · El acumulador que no acumula

```
total = 0
for n in [4, 7, 9]:
    total = n
print("Total:", total)
```

Salida: `Total: 9`, cuando 4 + 7 + 9 son 20.

Qué pasa: en cada vuelta **sustituye** el total en lugar de sumarle. Al final se queda con el último.

Corrección: `total = total + n`, o su forma abreviada `total += n`.

Cómo se detecta: punto de ruptura dentro del bucle y mirar `total` en cada vuelta: va 4, 7, 9 en vez de 4, 11, 20. **Este es exactamente el primer fallo de la Fase 3.**

4.2.5 Caso 11 · El que se deja el último

```
for i in range(1, 10):
    print(i, end=" ")
```

Salida: `1 2 3 4 5 6 7 8 9` — falta el 10.

Qué pasa: `range(1, 10)` llega **hasta 10 sin incluirlo**. Es así a propósito en Python, y sorprende siempre.

Corrección: `range(1, 11)`.

Cómo se detecta: contando el resultado. A este fallo se le llama *error de uno en uno* (*off-by-one*) y es uno de los más comunes que existen en programación.

Idea clave

Lo que hay que llevarse de esta parte:

Tipo de error	¿Te avisa alguien?	Con qué se encuentra
De sintaxis	Sí, el editor y el traductor	Leyendo el mensaje
De ejecución	Sí, al reventar	Leyendo el mensaje y el número de línea
Lógico	**No**	**El depurador y probar con casos conocidos**

Y la conclusión que enlaza con el resto del módulo: **que un programa se ejecute sin errores no significa absolutamente nada sobre si está bien.**